

Discrete Mathematics, Week 5

Graph Theory & Modeling

JeYoung Park

SeoulTech CS&CE

2026-05-07

Part 1 — 그래프 기초(Graph Basics)

- 그래프란? 왜 배우나? — $G = (V, E)$ 의 정의
- Directed vs Undirected, Simple/Multi/Pseudo
- 차수(Degree)와 Handshake 정리
- 특수 그래프: K_n , C_n , P_n , W_n , $K_{m,n}$, Q_n

Part 2 — 그래프 표현(Representation)

- 인접 행렬, 인접 리스트, 엣지 리스트 — 언제 무엇을?

Part 3 — 경로 & 연결성(Connectivity)

- Walk / Path / Cycle, 연결 성분, 강 / 약 연결

Part 4 — 모델링과 특수 그래프

- Bipartite & Matching, Eulerian / Hamiltonian, Isomorphism

지난 주에 배운 것

- 관계 $R \subseteq A \times A \Leftrightarrow$ 유향 그래프
- Boolean 행렬 M_R — 관계 행렬
- 동치관계 $\sim \Leftrightarrow$ 분할(Partition)
- Hasse diagram (DAG의 한 모습)

이번 주에 일반화

- 관계 행렬 $\rightarrow$ 인접 행렬
- 유향 그래프 $\rightarrow$ 일반 그래프
- 동치 부류 $\rightarrow$ 연결 성분
- 위상정렬 $\rightarrow$ DAG 위의 알고리즘

중심 질문

“대상들 사이의 **쌍-관계**를 통일된 언어 (그래프) 로 다루고, 거기에 깔린 **구조**로 어떻게 문제를 풀 것인가?”

Part 1

Graph Basics

정의 — 그래프라는 언어를 익히자

그래프(Graph) — 왜 배우나?

컴퓨터공학 곳곳에 있는 그래프

- **인터넷:** 라우터(정점) + 링크(간선) $\Rightarrow$ shortest path
- **소셜 네트워크:** 사용자(정점) + 친구관계(간선) $\Rightarrow$ community detection
- **웹:** 페이지(정점) + 하이퍼링크(유향 간선) $\Rightarrow$ PageRank
- **컴파일러:** 코드 블록(정점) + 점프(간선) $\Rightarrow$ CFG, dominator tree
- **빌드 시스템:** 파일(정점) + 의존성(간선) $\Rightarrow$ DAG, topological sort
- **블록체인:** 트랜잭션(정점) + 입출력(간선) $\Rightarrow$ UTXO graph

한 줄로

“서로 다른 대상 사이의 쌍-관계”라는 추상화 자체가 그래프이다.

그래프(Graph) — 정의

정의 1 (Graph)

그래프는 두 집합의 쌍 $G = (V, E)$ 이다.

- V : 정점(vertex)들의 집합 — 보통 유한, $|V| = n$
- E : 간선(edge)들의 집합 (V 의 2-부분집합 또는 순서쌍)

Directed vs Undirected

- **무향 그래프**: $E \subseteq \{ \{u, v\} : u, v \in V, u \neq v \}$ $\{u, v\}$ 는 “순서 없는 쌍”
- **유향 그래프(digraph)**: $E \subseteq V \times V$ (u, v) 는 “순서 있는 쌍” — $u \rightarrow v$

Week 3 와의 관계

유향 그래프는 집합 V 위의 이항관계와 **똑같은 대상**이다.

$E \subseteq V \times V \Leftrightarrow \text{relation}$. 새 이름, 같은 수학.

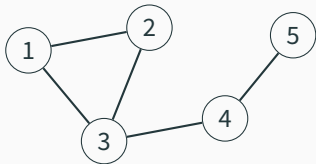
무향 그래프 예시

설정

$V = \{1, 2, 3, 4, 5\}$

$E =$

$\{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}, \{4, 5\}\}$



읽는 법

- 1 과 2 는 인접(adjacent)
- 5 는 4 하고만 인접
- $\{u, v\} \in E \Rightarrow$ “u와 v는 이웃”

핵심

무향 그래프의 간선은 “양방향 연결선” — 화살표 머리가 없다.

유향 그래프 예시

설정

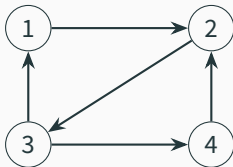
$V = \{1, 2, 3, 4\}$

$E =$

$\{(1, 2), (2, 3), (3, 1), (3, 4), (4, 2)\}$

읽는 법

- $(u, v) \in E: u \rightarrow v$ 화살표
- $1 \rightarrow 2$ 있어도 $2 \rightarrow 1$ 은 없을 수 있음
- Self-loop (v, v) 도 허용 가능



관계와의 일치

이 그래프 = 관계 $R = \{(1, 2), (2, 3), (3, 1), (3, 4), (4, 2)\}$.

Week 3의 “유향 그래프 표현”과 **글자 그대로** 같다.

Simple, Multi, Pseudo — 세 가지 그래프 종류

Simple Graph (단순 그래프)

- Self-loop **없음**: $(v, v) \notin E$
- Multi-edge **없음**: 같은 쌍에 간선 최대 1개
- 가장 흔한 기본 모델 — 별다른 말 없으면 “simple”

Multigraph (다중 그래프)

같은 두 정점 사이에 **여러 간선** 허용.

예: 두 도시 사이 항공 노선이 여러 개 (대한항공, 아시아나, ...)

Pseudograph (의사 그래프)

Self-loop 까지 허용.

예: 웹페이지가 자기 자신을 링크하는 경우.

Multi vs Pseudo — 그림으로



Simple



Multigraph



Pseudograph

이번 주 기본 가정

별다른 언급이 없으면 **simple, undirected** 그래프로 가정한다.

차수(Degree) — 정의

정의 2 (Degree — 무향)

정점 v 의 차수 $\deg(v)$ 는 v 에 닿은 간선의 수.

Self-loop 는 **2**로 센다 (양 끝이 모두 v 이므로).

정의 3 (In-degree, Out-degree — 유향)

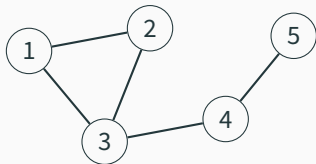
유향 그래프에서:

- $\deg^-(v)$: v 로 **들어오는** 간선 수
- $\deg^+(v)$: v 에서 **나가는** 간선 수

왜 차수를 보나?

- “인기도”: 페이스북 친구 수, 웹페이지 in-degree (PageRank의 출발점)
- “의존성 부담”: 빌드 그래프에서 in-degree 큰 모듈은 변경 위험
- 그래프 알고리즘의 **1-등급 통계량**

차수 계산 예시



각 정점의 차수

- $\deg(1) = 2$ (이웃: 2, 3)
- $\deg(2) = 2$ (이웃: 1, 3)
- $\deg(3) = 3$ (이웃: 1, 2, 4)
- $\deg(4) = 2$ (이웃: 3, 5)
- $\deg(5) = 1$ (이웃: 4)

합계

$$\sum_v \deg(v) = 2 + 2 + 3 + 2 + 1 = 10$$
$$\text{간선 수 } |E| = 5 \Rightarrow \text{합} = 2|E|.$$

관찰

차수 합이 간선 수의 **정확히 두 배**이다. 우연이 아니다 — Handshake Theorem.

Handshaking Lemma

Theorem 1 (Handshaking Lemma)

임의의 무향 그래프 $G = (V, E)$ 에서

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Proof — “각 간선이 두 번 세진다”

간선 $\{u, v\}$ 하나는 $\deg(u)$ 에 1, $\deg(v)$ 에 1 기여 $\Rightarrow$ 합에 **2** 기여.
간선 $|E|$ 개니까 총 기여 = $2|E|$. ■

Corollary 1 (홀수 차수 정점은 짝수 개)

$\sum \deg(v)$ 가 짝수($= 2|E|$)이므로, 홀수 차수 정점의 개수도 **짝수**.
 $\Rightarrow$ “홀수 명과 악수한 사람은 항상 짝수 명”이라는 파티 퀴즈.

Theorem 2 (Directed Handshake)

유향 그래프 $G = (V, E)$ 에서

$$\sum_{v \in V} \deg^+(v) = \sum_{v \in V} \deg^-(v) = |E|.$$

Proof

간선 (u, v) 하나는 $\deg^+(u)$ 에 1, $\deg^-(v)$ 에 1 기여.

한 번씩만 기여하므로 합 = $|E|$. ■

주의 — 무향과의 차이

무향: $2|E|$. 유향: $|E|$. 헷갈리지 말 것 — “간선이 두 번 세지는가?”로 기억.

왜 “이름 붙은 그래프”를 외우나?

구조가 자주 등장 + 알고리즘의 **기준 사례(benchmark)** 로 쓰인다.
이름을 알면 “아 이건 K_5 잤아” 하고 한 줄로 소통 가능.

앞으로 볼 6가지

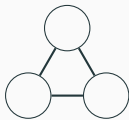
- K_n : 완전 그래프(complete) — “모두가 모두와 연결”
- C_n : 사이클(cycle) — n 개 정점이 한 바퀴
- P_n : 경로(path) — 일직선
- W_n : 바퀴(wheel) — C_n + 가운데 허브
- $K_{m,n}$: 완전 이분(bipartite complete)
- Q_n : n 차원 큐브(hypercube)

K_n — 완전 그래프(Complete)

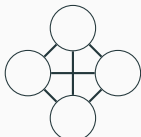
정의 4 (K_n)

n 개의 정점 **모든 쌍** 사이에 간선이 있는 simple 그래프.

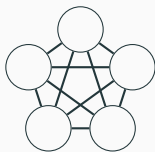
$$|V| = n, \quad |E| = \binom{n}{2} = \frac{n(n-1)}{2}.$$



K_3



K_4



K_5

차수

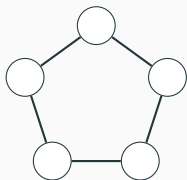
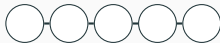
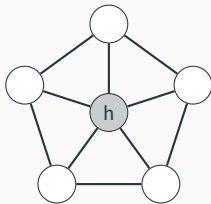
모든 정점의 $\deg(v) = n - 1$.

왜 중요?

- “최대 연결성”의 기준
- Ramsey 이론
- K_5 , $K_{3,3}$: 비평면성 판정

정의

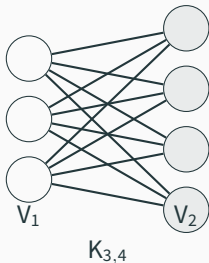
- C_n ($n \geq 3$): 정점 $v_1, \dots, v_n$ 이 **한 사이클**을 이룸.
 $|E| = n$, 모두 $\deg = 2$.
- P_n ($n \geq 1$): 정점 $v_1, \dots, v_n$ 이 **일직선**.
 $|E| = n - 1$, 양 끝 $\deg = 1$, 내부 $\deg = 2$.
- W_n ($n \geq 3$): C_n 의 가운데에 허브 정점 하나, 모든 정점에 간선.
 $|E| = 2n$, 허브 $\deg = n$, 외곽 $\deg = 3$.

 C_5  P_5  W_5

$K_{m,n}$ — 완전 이분 그래프

정의 5 (Bipartite complete)

정점 집합을 분할: V_1, V_2 with $V_1 \cup V_2 = V, V_1 \cap V_2 = \emptyset, |V_1| = m, |V_2| = n$. V_1 의 정점과 V_2 의 정점 사이의 모든 쌍에 간선, 같은 부분집합 안에는 간선 없음. $|E| = mn$.



쓰임새

- 매칭: 작업자-작업
- 추천 (사용자-아이템)
- DB join, network flow

비-예시

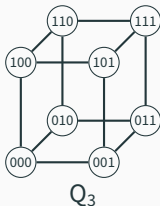
K_3 을 어떻게 두 부분집합으로 나누든 같은 부분집합 안에 간선이 생긴다
 $\Rightarrow$ 삼각형은 이분 그래프가 아니다.

정의 6 (Hypercube Q_n)

정점: n 비트 이진 문자열 $\{0, 1\}^n$ (2^n 개).

간선: 정확히 **한 비트만 다른** 두 문자열 사이.

$$|V| = 2^n, \quad |E| = n \cdot 2^{n-1}, \quad \deg(v) = n.$$



왜 CS에서 중요한가?

- **병렬 컴퓨팅**: hypercube 토폴로지의 통신 네트워크
- Gray code 와 동치
- Error-correcting codes 분석
- 비트 뒤집기 = 한 칸 이동

Subgraph — 부분 그래프

정의 7 (Subgraph)

$G = (V, E)$ 의 **subgraph** 는 $H = (V', E')$ 로서 다음을 모두 만족: $V' \subseteq V$, $E' \subseteq E$, 그리고 모든 $\{u, v\} \in E'$ 에 대해 $u, v \in V'$.

Induced subgraph (유도 부분 그래프)

$V' \subseteq V$ 만 정해도, 양 끝점이 V' 에 있는 **모든 간선**을 자동으로 포함:

$$E' = \{ \{u, v\} \in E : u \in V', v \in V' \}.$$

“정점만 고르면 간선이 자동 결정”되는 부분 그래프.

Clique — 모두가 모두에게 인접

$S \subseteq V$ 가 **clique** $\Leftrightarrow$ S 위의 induced subgraph 가 $K_{|S|}$. $|S| = k$ 면 **k-clique**. 예: 4-clique = “서로 모두 인접한 정점 4개”.

Part 2

Graph Representation

같은 그래프, 세 가지 데이터 구조

왜 “표현”을 따로 배우나?

같은 그래프, 다른 비용

- 인접 행렬: “ $u-v$ 인접인가?”는 $O(1)$.
하지만 “ v 의 모든 이웃 나열”은 $O(n)$ — 실제 이웃이 적어도 **행 전체 n 칸을 다 봐야 함**.
- 인접 리스트: “ v 의 이웃 나열”은 **이웃 개수만큼만** ($O(\deg v)$).
하지만 “ $u-v$ 인접인지” 검사하려면 $O(\deg v)$ 걸림.

선택 기준

- **Dense (밀집)** 그래프 ($|E| = \Theta(n^2)$): 인접 행렬
- **Sparse (희소)** 그래프 ($|E| = O(n)$): 인접 리스트
- **한 번만 읽고 끝**: 엣지 리스트

경험적 사실

실세계 그래프 (소셜, 웹, 도로)는 거의 다 **sparse** $\Rightarrow$ 인접 리스트가 기본.

인접 행렬(Adjacency Matrix)

정의 8 (Adjacency Matrix)

$|V| = n$. A 는 $n \times n$ 행렬, 정점 순서를 고정한 후

$$A[i][j] = \begin{cases} 1 & \{v_i, v_j\} \in E \quad (\text{또는 } (v_i, v_j) \in E) \\ 0 & \text{아니면} \end{cases}$$

무향 그래프이면 $A = A^T$ (대칭). Self-loop 있으면 $A[i][i] = 1$.

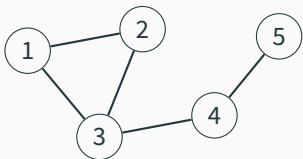
Week 3 와의 연결 — 동일!

이건 정확히 **관계 행렬** M_R . 행렬곱($\odot$)으로 길이- k 경로 수 계산도 그대로.
 $A^k[i][j]$ = i 에서 j 로 가는 길이 k 경로 개수 (Boolean이면 존재여부).

공간 복잡도

$O(n^2)$ 비트. $n = 10^5$ 면 행렬만으로 ~ 1.25 GB — 큰 그래프엔 부적합.

인접 행렬 예시



인접 행렬 A

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

관찰

- $A = A^T$ (무향이라 대칭)
- 행 i의 합 = $\deg(v_i)$

인접 리스트(Adjacency List)

정의 9 (Adjacency List)

각 정점 v 마다 **이웃들의 리스트**를 따로 저장.

$\text{adj}[v] = \{ u : \{v, u\} \in E \}.$

공간 복잡도

무향: $\sum_v \deg(v) = 2|E| \Rightarrow$ 총 $O(n + m)$ ($m = |E|$).

Sparse 그래프 ($m \ll n^2$)에서는 인접 행렬 $O(n^2)$ 보다 **훨씬 작다**.

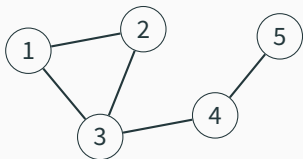
C++ 표현 (가장 흔한 형태)

```
vector<vector<int>>> adj(n);
```

```
adj[u].push_back(v);    adj[v].push_back(u);
```

모든 BFS/DFS/Dijkstra 의 **기본** 자료구조.

인접 리스트 예시



adj

- $\text{adj}[1] = [2, 3]$
- $\text{adj}[2] = [1, 3]$
- $\text{adj}[3] = [1, 2, 4]$
- $\text{adj}[4] = [3, 5]$
- $\text{adj}[5] = [4]$

공간

원소 수 = $2|E| = 10 \Rightarrow$ 행렬의 25 칸보다 훨씬 적다 — 그래프 커질수록 차이 폭증.

인접 행렬 vs 인접 리스트 — 비교표

연산	인접 행렬	인접 리스트
인접 여부 $\text{adj?}(u, v)$	$O(1)$	$O(\deg u)$
이웃 순회 $\text{neighbors}(u)$	$O(n)$	$O(\deg u)$
간선 추가/삭제	$O(1)$	$O(\deg u)$
공간	$O(n^2)$	$O(n + m)$
A^k (길이 k 경로)	행렬곱으로 자연	어색

Rule of thumb

m/n^2 이 클수록 (dense) 행렬, 작을수록 (sparse) 리스트.
실세계 그래프에선 **거의 항상 리스트**가 답.

엣지 리스트 & 가중 그래프

엣지 리스트 (Edge List)

간선들을 (u,v) 또는 (u,v,w) 의 단순 배열로 저장.

용도: Kruskal MST, 한 번 정렬 후 처리, 입출력 단계.

Weighted graph (가중 그래프)

간선마다 가중치 $w : E \rightarrow \mathbb{R}$ 부여.

- 행렬: $A[i][j] = w$ if $(i,j) \in E$, else ∞ (또는 0).
- 리스트: $\text{adj}[u] = [(v, w), \dots]$.

왜 가중을 도입하나?

도로의 길이, 네트워크 지연, 비용, 확률, 우정의 강도 ... “쌍-관계의 강도”까지 모델링하려면 가중치가 필수.

Part 3

Walks, Paths, Connectivity

그래프 위에서 “움직인다”는 것

Walk, Path, Cycle — 정의

정의 10 (Walk)

v_0 에서 v_k 로의 **walk(걸음)**: 정점-간선 교대 수열 $v_0, e_1, v_1, e_2, \dots, e_k, v_k$. 각 $e_i = \{v_{i-1}, v_i\} \in E$.
길이 = 간선 수 k . 정점/간선 **반복 허용**.

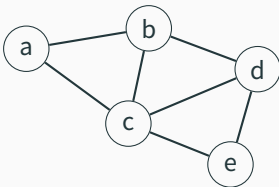
정의 11 (Trail / Path / Circuit / Cycle)

- **Trail**: 간선이 모두 다른 walk
- **Path**: 정점도 모두 다른 walk
- **Circuit**: $v_0 = v_k$ 인 trail (닫힌 trail)
- **Cycle**: $v_0 = v_k$ 이고 그 외 정점은 모두 다른 walk

한 줄 요약

Walk 는 자유, Trail 은 간선 비반복, Path 는 정점도 비반복.

Walk vs Path — 그림으로



예 1 — Walk (정점/간선 반복 OK)

$a - b - c - a - c - d$ (a, c 두 번씩, 길이 5)

예 2 — Trail (간선 비반복)

$a - b - c - d - e - c$ (정점 c 반복, 간선은 모두 다름)

예 3 — Path (정점 비반복)

$a - b - d - e$ (모두 다른 정점, 길이 3)

길이 k walk 개수 = A^k

Theorem 3 (Walk Counting)

A가 그래프 G의 인접 행렬일 때, i에서 j로의 **walk** 개수는

$$(A^k)_{ij} \quad (\text{단, } k = \text{walk 길이}).$$

Proof — 수학적 귀납법

Base ($k = 1$): A_{ij} 는 길이 1 walk 개수 (간선 있으면 1, 없으면 0). ✓

Step: A_{ij}^k 가 길이 k walk 수라 가정. 행렬곱:

$$A_{ij}^{k+1} = \sum_l A_{il}^k A_{lj}.$$

l을 k번째 정점으로 보면, $i \rightarrow \dots \rightarrow l \rightarrow j$ walk의 합. ■

Boolean과의 차이

일반 행렬곱: **개수**. Boolean($\vee, \wedge$): **존재여부**. Warshall의 토대.

연결성(Connectivity) — 무향

정의 12 (Connected)

무향 그래프 G 가 **연결(connected)**이다 $\Leftrightarrow$ 임의의 두 정점 u, v 사이에 **경로(path)**가 존재.

정의 13 (Connected Component)

“ $u \sim v \Leftrightarrow u, v$ 사이에 path 존재”라는 관계는 **동치관계** (Week 3에서 봤듯이 reflexive/symmetric/transitive).

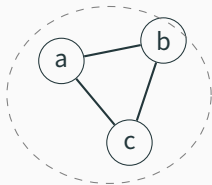
이 동치관계의 동치 부류를 연결 성분이라 부른다.

Week 3의 그 정리가 여기서 작동

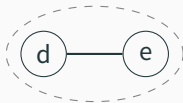
“동치관계 $\Leftrightarrow$ 분할” (Theorem 3, Week 3) $\Rightarrow$

모든 무향 그래프의 정점 집합 V 는 연결 성분으로 완전 분할된다.

연결 성분 — 예시



comp 1



comp 2



comp 3

이 그래프의 연결 성분

$\{a, b, c\}$, $\{d, e\}$, $\{f\}$ — 세 개. **덮음 + 서로소**, 분할 조건 만족.

알고리즘으로 찾기

DFS / BFS 한 번이면 한 성분 발견. 모든 정점 방문할 때까지 반복 $\Rightarrow O(n + m)$.

또는 Union-Find 로 동적으로 관리 (Week 3 에서 예고한 자료구조).

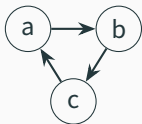
유향 그래프의 연결성 — 두 종류

Weakly Connected (약연결)

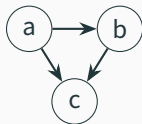
화살표 방향을 무시하고 무향으로 만든 후 connected 면 weakly connected.

Strongly Connected (강연결)

임의의 u, v 에 대해 $u \rightarrow \dots \rightarrow v$ 와 $v \rightarrow \dots \rightarrow u$ 모두 존재.



Strongly (사이클 존재)



Weakly만 ($a \rightarrow b \rightarrow c$, 역 X)

관계는?

강연결 $\Rightarrow$ 약연결. 역은 거짓. **DAG**는 강연결 불가능 (사이클 0).

Strongly Connected Component (SCC)

정의 14 (SCC)

“ $u \leftrightarrow v \Leftrightarrow$ 양방향 도달 가능”은 동치관계.

이 관계의 동치 부류 = **strongly connected component**.

SCC 응축(condensation)

모든 SCC 를 하나의 “초정점”으로 합치면 결과 그래프는 **DAG**.

$\Rightarrow$ 어떤 유형 그래프든 “DAG of SCCs”로 분해 가능.

알고리즘 (다음 학기 알고리즘 수업 예고)

- Kosaraju: DFS 두 번 + 그래프 뒤집기, $O(n + m)$
- Tarjan: DFS 한 번, lowlink, $O(n + m)$

Part 4

Modeling & Special Graphs

문제를 그래프로 번역하기

모델링이란?

현실 문제를 “V 는 무엇? E 는 무엇?” 두 줄로 **번역**하는 작업.
잘 번역하면 자료구조와 알고리즘이 곧장 따라온다.

4가지 표준 패턴

- **State graph**: 정점 = 상태, 간선 = 한 단계 전이.
⇒ 퍼즐, 게임, 자동기계.
- **Constraint graph**: 정점 = 변수, 간선 = 두 변수 사이 제약.
⇒ 시간표, 색칠, SAT.
- **Dependency graph**: 정점 = 작업, 간선 = “먼저 해야”.
⇒ DAG, topological sort.
- **Network graph**: 정점 = 거점, 간선 = 채널/도로/회선.
⇒ shortest path, max flow.

모델링 사례 1 — 지도 색칠

문제

지도 위 **인접한 두 나라는 다른 색**으로 칠하라. 최소 색 수는?

그래프 번역

- V: 나라 하나 = 정점 하나
- E: 두 나라가 국경을 공유하면 간선 (무향)

원래 문제 $\Leftrightarrow$ **Graph Coloring**: 인접한 두 정점이 다른 색을 갖도록 색칠.
Chromatic number $\chi(G)$ = G 를 적절히 색칠하는 데 필요한 최소 색 수.

사실(언급만): 4-Color Theorem

지도에서 나오는 그래프(planar)는 항상 **4색**으로 충분.
1976 년 컴퓨터 보조 증명 — 수학사 최초의 “컴퓨터 증명”.

모델링 사례 2 — 시간표 짜기

문제

강의 여러 개와 “동시 수강 학생” 이 있을 때, 같은 시간대에 두 강의를 못 잡아 **충돌**을 피하는 시간 슬롯 배정.

그래프 번역

- V: 강의 하나 = 정점 하나
- E: 두 강의에 **공통 학생**이 있으면 간선
- 색 = 시간 슬롯

시간표 = 정점 색칠. $\chi(G)$ 가 필요한 시간 슬롯의 수.

또 다른 해석

나라 색칠 = 시간표 짜기 = 레지스터 할당 (컴파일러) $\Rightarrow$ **같은 추상 문제** (graph coloring), 셋 다 NP-hard.

모델링 사례 3 — 의존성과 빌드 순서

문제

“a.o 는 a.c 와 a.h 가 필요. prog 는 a.o, b.o 필요. ...”
어떤 **순서**로 컴파일?

그래프 번역 (Week 3 의 직접 응용)

- V: 파일 / 작업
- E: $u \rightarrow v =$ “u 가 만들어져야 v 시작 가능”

사이클이 있으면 모순 (서로 먼저 끝내야 함) $\Rightarrow$ **DAG** 가정.

순서 = **topological sort**. Kahn 알고리즘 $O(n + m)$ (Week 3, p. 89).

현실 사례

make, ninja, cargo, npm, nix, 강의 선수 과목, 영화 시나리오 — 모두 같은 패턴.

모델링 사례 4 — 최단 경로

문제

도시 s 에서 도시 t 까지 **가장 빠른** 길? 도로는 시간이 다 다름.

그래프 번역

- V : 도시
- E : 도로, 가중치 $w(e)$ = 통과 시간
- 답: 가중치 합이 최소인 $s-t$ path

⇒ **Single-source shortest path.**

$w \geq 0$: Dijkstra $O((n + m) \log n)$. 음수 가중: Bellman-Ford $O(nm)$.

같은 모델, 다른 의미

“시간” 대신 비용, 거리, 환승 횟수, 패킷 손실 확률의 $-\log \dots$

“가장 좋은 경로” 류 문제는 거의 다 가중 그래프 + shortest path.

Bipartite Graph — 정의와 판정

정의 15 (Bipartite)

$G = (V, E)$ 가 **이분 그래프**이다 $\Leftrightarrow$ 정점 집합 V 를 두 부분집합 V_1, V_2 로 분할 ($V_1 \cup V_2 = V, V_1 \cap V_2 = \emptyset$, 둘 다 비지 않음) 하여 모든 간선이 V_1 의 정점과 V_2 의 정점을 잇는 형태로만 존재.

Theorem 4 (이분 판정)

G 가 이분 $\Leftrightarrow G$ 에 **홀수 길이 사이클이 없다**.

알고리즘 — BFS 2색칠

- 시작 정점에 색 0 부여, BFS 진행하며 이웃에 **반대색** 부여
- 이웃이 이미 **같은 색**이면 충돌 $\Rightarrow$ 이분 아님
- $O(n + m)$

왜 “홀수 사이클”인가?

사이클을 따라 색이 0,1,0,1,...교대. 사이클을 한 바퀴 돌고 출발점에 돌아오려면 짝수 길이어야 함. 홀수면 모순.

Matching — “짝짓기 문제”

정의 16 (Matching)

$M \subseteq E$ 가 **매칭** $\Leftrightarrow M$ 의 어떤 두 간선도 **끝점을 공유하지 않음**. Maximum matching: $|M|$ 최대인 매칭.

이분 그래프에서의 매칭 ($V = V_1 \cup V_2$)

- $V_1 =$ 작업자, $V_2 =$ 작업 $\Rightarrow$ **최대 배정**
- $V_1 =$ 학생, $V_2 =$ 동아리 $\Rightarrow$ **최대 가입**
- $V_1, V_2 =$ 결혼 가능 남/여 (“결혼 정리”)

$O(\sqrt{n}m)$ Hopcroft-Karp 알고리즘 (다음 학기).

Hall's theorem (이름만)

$S \subseteq V_1$ 의 이웃 집합 $N(S) = \{v \in V_2 : \exists u \in S, \{u, v\} \in E\}$ 라 하자. V_1 전체를 매칭 가능 $\Leftrightarrow \forall S \subseteq V_1, |N(S)| \geq |S|$. “병목 없으면 매칭 성공.”

Eulerian Trail / Circuit — 직관

역사 — 쾨니히스베르크 다리 문제(1736)

강과 4 구역, 7개 다리. “각 다리를 **정확히 한 번씩** 건너 출발지로 돌아올 수 있나?”

오일러: **불가능**. 그래프 이론의 출발점.

정의 17 (Eulerian)

- **Eulerian trail**: 그래프의 모든 간선을 **정확히 한 번씩** 지나는 trail.
- **Eulerian circuit**: 시작점에 돌아오는 Eulerian trail.

Theorem 5 (Euler)

연결 그래프 G 에서:

- Eulerian **circuit** 존재 $\Leftrightarrow$ 모든 정점의 차수가 짝수.
- Eulerian **trail** 존재 $\Leftrightarrow$ 홀수 차수 정점이 정확히 0개 또는 2개.

Euler 정리 — 직관 Proof

circuit 의 “ $\Rightarrow$ ” (필요조건)

Circuit 이 정점 v 를 거칠 때마다 **들어오는 간선 1개 + 나가는 간선 1개** 사용.

v 를 k 번 지나면 그곳에 닿은 간선 $2k$ 개를 사용 $\Rightarrow \deg(v)$ 가 짝수.

trail 의 두 끝점

출발점과 도착점만 “들어오기/나가기 짝수 짝짓기”가 깨짐.

$\Rightarrow$ 홀수 차수 정점은 **0개(=circuit)** 또는 **정확히 2개(=trail의 두 끝점)**.

코니히스베르크 결론

4 구역의 차수: 3, 3, 3, 5 $\Rightarrow$ **홀수 차수 4개** $\Rightarrow$ trail 도 circuit 도 불가능.

Hamiltonian — 모든 정점을 한 번씩

정의 18 (Hamiltonian)

- **Hamiltonian path**: 모든 정점을 **정확히 한 번씩** 지나는 path.
- **Hamiltonian cycle**: 시작점으로 돌아오는 Hamiltonian path.

Eulerian과의 차이 — 헷갈리지 말기

- Eulerian: **간선**을 다 한 번씩 (다항 시간 판정 가능, 차수만 보면 됨)
- Hamiltonian: **정점**을 다 한 번씩 (**NP-complete**, 다항 알고리즘 미존재 추정)

“간선 vs 정점”의 한 글자 차이가 **P vs NP**의 차이.

유명 사례 — TSP

Travelling Salesperson Problem = 가중 Hamiltonian cycle 의 최소 비용.

조합 최적화의 “별”.

Graph Isomorphism — “같은 그래프 다른 그림”

정의 19 (Isomorphism)

$G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$ 가 **isomorphic** ($G_1 \cong G_2$) $\Leftrightarrow$ 존재하는 **bijection** $f: V_1 \rightarrow V_2$ 가 있어

$$\{u, v\} \in E_1 \Leftrightarrow \{f(u), f(v)\} \in E_2.$$

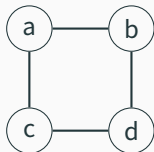
Week 3 의 bijection 이 여기서 작동

“간선 보존하는 bijection” — Week 3 함수 파트의 **직접 활용**.

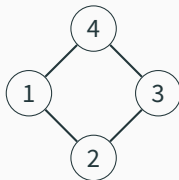
Invariants — 빠른 “아닌지” 검사

서로 isomorphic 이면 다음이 모두 같다: $|V|$, $|E|$, 차수 multiset(정렬된 차수열), 사이클 길이 multiset, 연결 성분 수, 지름 ... 하나라도 다르면 **이미 isomorphic 아님**이 확정.

Isomorphism 예시



G_1 (사각형)



G_2 (마름모)

$G_1 \cong G_2$?

차수 multiset: $G_1 = \{2, 2, 2, 2\}$, $G_2 = \{2, 2, 2, 2\}$. ✓

간선 수 4, 정점 수 4 일치. bijection $f: a \rightarrow 1, b \rightarrow 2, d \rightarrow 3, c \rightarrow 4$
가 간선을 보존 $\Rightarrow$ **Yes, isomorphic.**

둘 다 사실 C_4 .

계산 복잡도 — 미해결

일반 그래프 isomorphism 의 다항 알고리즘 존재 여부는 **미해결**.

2017 Babai의 quasi-polynomial 알고리즘 ($n^{O(\log^c n)}$)이 현재 최선.

Tree — 다음 주 예고편

정의 20 (Tree)

연결되어 있고 **사이클이 없는** 무향 그래프.

Tree 의 동치 정의 (모두 같은 말)

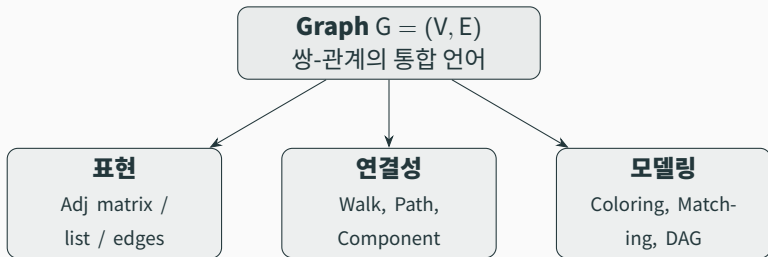
G가 n개 정점일 때 다음은 동치:

1. G 는 트리 / 두 정점 사이에 **유일한** path 존재
2. G 는 연결, $|E| = n - 1$ / G 는 사이클 없음, $|E| = n - 1$
3. Minimally connected (간선 하나 빼면 분리)

왜 미리?

BST, heap, trie, Merkle, Spanning tree (Kruskal/Prim), DFS tree, parse tree — **컴퓨터과학에서 가장 많이 쓰이는 그래프.**

Big Picture — 이번 주에 배운 것



Eulerian, Hamiltonian, Bipartite, Tree, Isomorphism

핵심 한 줄

관계 = 유형 그래프, 동치관계 = 연결 성분의 분할, Hasse = DAG. Week 3 가 한 줄로 일반화된다.

핵심 알고리즘 & 복잡도

문제	최선의 일반 복잡도
연결 성분 (DFS/BFS)	$O(n + m)$
이분 판정 (BFS 2-색)	$O(n + m)$
SCC (Tarjan, Kosaraju)	$O(n + m)$
Topological sort (Kahn)	$O(n + m)$
Shortest path, $w \geq 0$ (Dijkstra)	$O((n + m) \log n)$
MST (Kruskal/Prim)	$O(m \log n)$
Eulerian circuit 판정	$O(n + m)$
Maximum bipartite matching	$O(\sqrt{n} m)$
Hamiltonian path 판정	NP-complete
Graph coloring ($\chi(G)$)	NP-hard

관찰

간선 한 번씩만 보면 풀리는 문제는 **선형 시간**. “정점을 모두 다”(Hamiltonian, coloring)는 갑자기 **지수** 가능성.

Week 6: Trees

- Rooted tree, parent/child, depth, height
- Binary / m-ary tree, full / complete / balanced
- Tree traversal: pre/in/post-order, BFS-order
- Spanning tree, MST (Kruskal, Prim)
- Applications: BST, heap, trie, Huffman, Merkle, parse tree

이번 주가 다음 주의 토대

오늘의 **연결성 + 사이클 없음**이 트리 정의의 두 축. 오늘의 **가중 그래프 + 모델링** $\Rightarrow$ MST의 동기.

마무리 — 기억해야 할 5가지

1. $G = (V, E)$ 의 두 가지 표현: 인접 행렬 ($O(n^2)$, 밀집용), 인접 리스트 ($O(n + m)$, sparse용 – 실세계 기본값).
2. **Handshake**: $\sum \deg(v) = 2|E|$.
무향 1번에 두 번 세짐 / 유향은 in과 out이 각각 $|E|$.
3. **연결 성분 = 동치관계의 분할**.
Week 3 의 “동치 $\Leftrightarrow$ 분할”이 그래프 위에서 그대로 작동.
4. **모델링 4 패턴**: state, constraint, dependency, network.
문제를 “V 는?, E 는?” 두 줄로 번역할 수 있어야 함.
5. **Eulerian = 간선 한 번씩 (다항) vs Hamiltonian = 정점 한 번씩 (NP-c)**.
한 글자 차이가 P vs NP 차이. “간선이냐 정점이냐”로 기억.

수고하셨습니다! 질문 환영합니다.